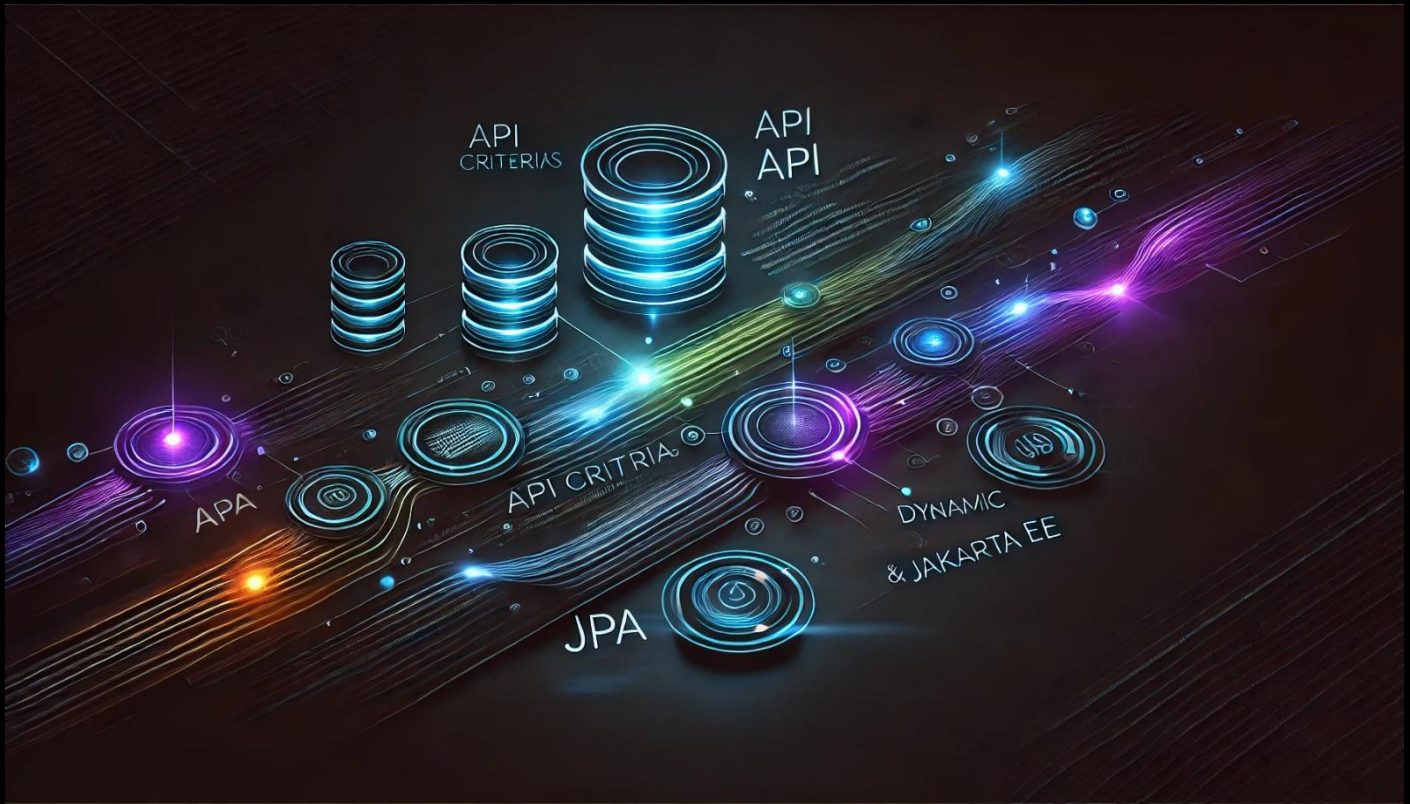


API de Criteria con JPA y Jakarta EE



Uso de la API de Criteria en JPA 🚀

En esta guía aprenderás a utilizar la **API de Criteria** en **JPA** para realizar consultas dinámicas de manera segura y eficiente. A diferencia de **JPQL**, donde las consultas se escriben como cadenas de texto, **Criteria** permite construirlas de forma programática, proporcionando **tipado seguro** y facilitando la generación de consultas más dinámicas.

1 Conceptos Básicos de la API de Criteria 📁

♦ ¿Qué es la API de Criteria?

La API de Criteria es un conjunto de clases en JPA que permite construir consultas de manera programática en Java. A diferencia de JPQL, donde las consultas se escriben en forma de cadenas, Criteria permite crear objetos de consulta con métodos y estructuras de Java.

♦ Ventajas del uso de Criteria en JPA:

- ✓ **Consultas dinámicas:** Permite generar consultas en tiempo de ejecución.
- ✓ **Tipado seguro:** Reduce errores al evitar escribir consultas en forma de texto.
- ✓ **Compatibilidad con JPQL:** Se basa en la misma estructura de consultas de JPQL.
- ✓ **Manejo de filtros condicionales:** Permite agregar condiciones dinámicamente sin concatenar strings.

◆ Sintaxis Básica de Criterias

```
// 1. Obtener el CriteriaBuilder desde el EntityManager
CriteriaBuilder cb = entityManager.getCriteriaBuilder();

// 2. Crear una consulta de Criteria para una entidad específica
CriteriaQuery<Persona> criteriaQuery = cb.createQuery(Persona.class);

// 3. Definir la raíz de la consulta (desde qué entidad se seleccionarán los datos)
Root<Persona> fromPersona = criteriaQuery.from(Persona.class);

// 4. Seleccionar los datos
criteriaQuery.select(fromPersona);

// 5. Ejecutar la consulta y obtener resultados
TypedQuery<Persona> query = entityManager.createQuery(criteriaQuery);
List<Persona> personas = query.getResultList();
```

2 Implementación Paso a Paso

Ahora aplicaremos la API de Criteria en JPA para realizar la consulta de **todas las personas** registradas en la base de datos.

3 Implementación en Código

```
package sga.cliente.criteria;

import java.util.List;
import jakarta.persistence.*;
import jakarta.persistence.criteria.*;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```
public class PruebaApiCriteria {

    private static final Logger log = LoggerFactory.getLogger(PruebaApiCriteria.class);

    public static void main(String[] args) {
        // 1. Crear la fábrica de entidades
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");

        try (EntityManager em = emf.createEntityManager()) {

            // 2. Crear el CriteriaBuilder
            CriteriaBuilder cb = em.getCriteriaBuilder();

            // 3. Crear una consulta de Criteria para la entidad Persona
            CriteriaQuery<Persona> criteriaQuery = cb.createQuery(Persona.class);

            // 4. Definir la raíz de la consulta (especificar de qué entidad obtener los datos)
            Root<Persona> fromPersona = criteriaQuery.from(Persona.class);

            // 5. Seleccionar los datos desde la raíz
            criteriaQuery.select(fromPersona);

            // 6. Crear y ejecutar la consulta
            TypedQuery<Persona> query = em.createQuery(criteriaQuery);
            List<Persona> personas = query.getResultList();

            // 7. Mostrar los resultados
            mostrarPersonas(personas);
        }
    }

    // Método para mostrar los resultados obtenidos
    private static void mostrarPersonas(List<Persona> personas) {
        for (Persona p : personas) {
            log.info("Persona: " + p);
        }
    }
}
```

4. Explicación del Código 🤖

1. Crear la fábrica de entidades

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
```

✚ Permite acceder a la base de datos y gestionar las entidades.

2. Crear el CriteriaBuilder

```
CriteriaBuilder cb = em.getCriteriaBuilder();
```

✚ `CriteriaBuilder` es la clase principal para construir consultas con `Criteria`.

3. Crear una consulta de Criteria

```
CriteriaQuery<Persona> criteriaQuery = cb.createQuery(Persona.class);
```

✚ Se crea un objeto `CriteriaQuery` para trabajar con la entidad `Persona`.

4. Definir la raíz de la consulta

```
Root<Persona> fromPersona = criteriaQuery.from(Persona.class);
```

✚ Define desde qué entidad se extraerán los datos (`FROM Persona` en SQL).

5. Seleccionar los datos desde la raíz

```
criteriaQuery.select(fromPersona);
```

✚ Se indica que la consulta seleccionará toda la entidad `Persona`.

6. Crear y ejecutar la consulta

```
TypedQuery<Persona> query = em.createQuery(criteriaQuery);  
List<Persona> personas = query.getResultList();
```

✚ Se ejecuta la consulta y se obtienen los resultados en una lista.

7. Mostrar los resultados

```
private static void mostrarPersonas(List<Persona> personas) {  
    for (Persona p : personas) {  
        log.info("Persona: " + p);  
    }  
}
```

✖ Se recorren los resultados y se imprimen en la consola.

5 Salida Esperada en la Consola 🖨

Si la base de datos tiene registros en la tabla `Persona`, verás un resultado similar a este:

```
[EL Fine]: sql: ServerSession(932257672)--Connection(1818747191)--SELECT id_persona, APELLIDO, EMAIL, NOMBRE,
TELEFONO FROM PERSONA
[main] INFO sga.cliente.criterio.PruebaApiCriterio - Persona: Persona{idPersona=1, nombre=Rafael,
apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}
[main] INFO sga.cliente.criterio.PruebaApiCriterio - Persona: Persona{idPersona=2, nombre=Ivonne,
apellido=Lara, email=ilara@mail.com, telefono=55443322}
[main] INFO sga.cliente.criterio.PruebaApiCriterio - Persona: Persona{idPersona=3, nombre=Maria,
apellido=Esparza, email=mesparza@mail.com, telefono=55112233}
[main] INFO sga.cliente.criterio.PruebaApiCriterio - Persona: Persona{idPersona=5, nombre=Hugo,
apellido=Hernandez, email=hhernandez@mail.com, telefono=55778822}
-
```

6 Conclusión 🎯

- ✅ Usamos la API de Criterio en lugar de JPQL para construir consultas programáticamente.
- ✅ Aprendimos los pasos clave para crear y ejecutar una consulta Criterio en JPA.
- ✅ Implementamos mejoras en el código para mayor claridad y reutilización.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)